

Principles of Programming Languages, 2025.09.10

Important notes

- Total available time: 2h (*multichance* students do not need to solve Exercise 3).
- You may use any written material you need, and write in English or in Italian.
- You cannot use electronic devices during the exam: every phone must be turned off and kept on your table.
- You cannot use library functions not covered in class in your code.
- Each exercise is valued 10 points, +1 bonus.

Exercise 1, Scheme

Define a pure function (without using side effects such as `set !`) which takes a multi-level list, i.e. a list that may contain any level of sublists, and converts it into a data structure where:

- Every single level **list** is converted into a **string**, formed by concatenating the string representations of its elements.
- If an element is itself a list, it is recursively converted.
- Non-string elements are converted to strings using `number->string` or `symbol->string` as appropriate.

For example: `(list->string-tree '(1 (2 3) (a (b c))))`
`("1" "23" ("a" "bc"))`

Exercise 2, Haskell

Define a data-type called **QTT** (for *Quaternary-Ternary Trees*), where nodes can be either *ternary* or *quaternary*, and each node contains a value. The empty tree `Nil` is also allowed. Note: there must not be unary or binary nodes.

1. Make QTT an instance of **Functor** and **Foldable**.
2. Define a binary **merge** operation for QTT, with the following constraints:
 - If one of the operands is a ternary node, it must become quaternary, and the other operand will become the fourth child.
 - If both operands are quaternary nodes, then the right operand must be merged into the rightmost child of the left operand (recursively).
3. Make QTT an instance of **Applicative**.

Exercise 3, Erlang

Consider a **main process** which takes a list of *tuples* of the form `{Fun, Params}`, where:

- `Fun` is a function (an atom bound in the current module).
- `Params` is the list of arguments for that function.

The main process must:

1. Spawn a **supervisor process** for each `{Fun, Params}` tuple.
2. Each supervisor process must spawn a worker running `Fun(Params)`.
3. If the worker dies for any reason, the supervisor must restart it **up to 3 times**. If it fails 3 times, the supervisor stops.
4. The main process finishes only when all supervisors terminate.

Solutions

Ex 1

```
(define (atom->string x)
  (cond ((number? x) (number->string x))
        ((symbol? x) (symbol->string x))
        ((string? x) x)
        (else (error "Unsupported type" x))))

(define (flat-list? lst)
  (or (not (list? lst))
      (null? (filter list? lst))))

(define (list->string-tree lst)
  (cond ((null? lst) "")
        ((and (list? lst) ;; flat-list
              (null? (filter list? lst)))
         (apply string-append (map atom->string lst)))
        ((list? lst)
         (map (lambda (elem)
                (if (list? elem)
                    (list->string-tree elem)
                    (atom->string elem)))
              lst))
        (else (atom->string lst))))
```

Ex 2

```
data QTT a
  = Nil
  | Ternary a (QTT a) (QTT a) (QTT a)
  | Quaternary a (QTT a) (QTT a) (QTT a) (QTT a)
  deriving (Show, Eq)

instance Functor QTT where
  fmap _ Nil = Nil
  fmap f (Ternary x l m r) =
    Ternary (f x) (fmap f l) (fmap f m) (fmap f r)
  fmap f (Quaternary x a b c d) =
    Quaternary (f x) (fmap f a) (fmap f b) (fmap f c) (fmap f d)

instance Foldable QTT where
  foldr _ z Nil = z
  foldr f z (Ternary x l m r) =
    f x (foldr f (foldr f (foldr f z r) m) l)
  foldr f z (Quaternary x a b c d) =
    f x (foldr f (foldr f (foldr f (foldr f z d) c) b) a)

merge :: QTT a -> QTT a -> QTT a
merge Nil t = t
merge t Nil = t

merge (Ternary x l m r) t =
  Quaternary x l m r t

merge t (Ternary x l m r) =
  Quaternary x t l m r

merge (Quaternary x a b c d) t =
  Quaternary x a b c (merge d t)

tconcat :: QTT (QTT a) -> QTT a
tconcat = foldr merge Nil

tconcmmap :: (a -> QTT b) -> QTT a -> QTT b
tconcmmap f t = tconcat $ fmap f t

instance Applicative QTT where
  pure x = Ternary x Nil Nil Nil
  fs <*> xs = tconcmmap (\f -> fmap f xs) fs
```

Ex 3

```
start(FunParamsList) ->
  Parent = self(),
  [spawn(?MODULE, supervisor, [Parent, Fun, Params, 3])
   || {Fun, Params} <- FunParamsList],
  wait_all(length(FunParamsList)),
  io:format("All supervisors finished.\n").

wait_all(0) -> ok;
wait_all(N) ->
  receive
    {done, _} -> wait_all(N - 1)
  end.

supervisor(Parent, Fun, Params, 0) ->
  io:format("Supervisor: ~p(~p) failed 3 times, giving up.\n", [Fun, Params]),
  Parent ! {done, self()};
supervisor(Parent, Fun, Params, Attempts) ->
  process_flag(trap_exit, true),
  Pid = spawn_link(?MODULE, worker, [self(), Fun, Params]),
  receive
    %% Success path (worker sends its result)
    {ok, Pid, Result} ->
      io:format("Worker ~p(~p) finished successfully: ~p~n",
        [Fun, Params, Result]),
      Parent ! {done, self()};
    %% If the worker exits normally before/after {ok,...} arrives
    {'EXIT', Pid, normal} ->
      io:format("Worker ~p(~p) finished successfully.\n", [Fun, Params]),
      Parent ! {done, self()};
    %% Crash path: restart
    {'EXIT', Pid, Reason} ->
      io:format("Worker ~p(~p) crashed with ~p. Restarting... (~p left)~n",
        [Fun, Params, Reason, Attempts - 1]),
      supervisor(Parent, Fun, Params, Attempts - 1)
  end.

worker(Supervisor, Fun, Params) ->
  Result = apply(?MODULE, Fun, Params),
  Supervisor ! {ok, self(), Result}.
```